DATA ENGINEER BOOTCAMP ADVANCED · CAPSTONE

Tugas Besar — Olist Data Platform

Dari 9 file CSV mentah sampai dashboard yang bisa dipertanggungjawabkan

Dataset: Brazilian E-Commerce Public Dataset by Olist (Kaggle) — 99.441 order, 9 file CSV, ±1,5 juta baris.

Stack: PostgreSQL · Kafka · MinIO · Jupyter (Local Big Data Lab) + Power BI/notebook.

Bentuk: tugas **individu** — 7 tahap wajib (T0–T6) + 3 bonus, dikerjakan bertahap, 3 checkpoint, demo 10 menit di Grand Demo Day.

Yang dinilai: kebenaran yang terbukti, idempotensi, ketahanan terhadap gangguan, dan keputusan desain yang bisa dipertahankan.

Bayuzen Ahmad · Fullstack Bangalore

Oktober 2026

Daftar isi

1. Gambaran tugas 3
1.1 Ceritanya 3
1.2 Yang kamu bawa pulang 3
1.3 Aturan main 3
2. Dataset 5
2.1 Mengunduh 5
2.2 Sembilan file 5
2.3 Yang perlu kamu tahu sebelum menyentuh apa pun 5
3. Arsitektur target 7
4. Peta tahap 8
5. Tahap demi tahap 9
T0 — Kontrak data & profil 9
T1 — Bronze: ingest ke MinIO 9
T2 — Silver + raw + gate kualitas 10
T3 — Streaming replay di Kafka 11
T4 — Star schema (dw) 13
T5 — Mart & dashboard 14
T6 — Failure test, evidence, runbook, demo 14
6. Bonus (maks +15, dinilai hanya jika T0–T6 lengkap) 16
7. Deliverables & struktur repository 17
8. Penilaian 18
8.1 Rubrik 18
8.2 Angka cek 18
8.3 Demo 10 menit (Grand Demo Day) 18
9. Pertanyaan yang sering muncul 20

1. Gambaran tugas

1.1 Ceritanya

Olist adalah marketplace di Brasil yang menghubungkan toko-toko kecil dengan pembeli di seluruh negeri. Dataset publiknya (Kaggle, 2016–2018) berisi **99.441 order** lengkap dengan item, pembayaran, review, pelanggan, seller, dan koordinat geografis — sembilan file CSV, sekitar 1,5 juta baris.

Bayangkan kamu baru masuk sebagai *data engineer* di tim data Olist. Manajemen punya empat pertanyaan yang sampai sekarang dijawab dengan Excel dan tebakan:

1. **Pendapatan** — berapa pendapatan per bulan dan per kategori, dan apakah angka itu cocok dengan uang yang benar-benar dibayar?
2. **Pengiriman** — di negara bagian mana pengiriman paling sering terlambat dari estimasi, dan apa dampaknya ke skor review?
3. **Seller** — seller mana yang perlu ditindak karena sering terlambat, sering dibatalkan, atau review-nya buruk?
4. **Pelanggan** — siapa pelanggan kita: berapa yang belanja ulang, siapa yang bernilai tinggi tapi sudah lama tidak kembali?

Dan satu permintaan dari tim operasional: **feed status order yang hampir real-time** (dibuat → disetujui → dikirim → sampai), yang tetap benar meskipun sistem hulu mengirim event dua kali atau tidak urut.

Tugasmu: membangun **platform data** yang menjawab semua itu — dari file mentah sampai dashboard — di atas stack yang sudah kamu kenal dari *Your Big Data Lab* (PostgreSQL, Kafka, MinIO, Jupyter dalam satu `docker compose`), dengan bukti bahwa platform itu **benar, bisa diulang, dan tahan gangguan**.

1.2 Yang kamu bawa pulang

Setelah tugas ini selesai kamu punya satu repository portofolio yang menunjukkan kamu bisa:

- menulis **kontrak data** dari dataset yang kotor (dan Olist memang kotor di tempat-tempat yang tidak terlihat),
- membangun **data lake** bertingkat (bronze → silver → gold) di object storage yang idempoten,
- memasang **gate kualitas** yang memisahkan "kotoran biasa" dari "sumber rusak",
- membangun **pipeline streaming** yang tahan duplikat, event terlambat, dan pesan rusak,
- memodelkan **star schema** yang grain-nya benar dan integritasnya terbukti,
- menyajikan **mart dan dashboard** yang tiap angkanya bisa ditelusuri ke tabel fakta,
- dan — ini yang membedakan engineer dari pembuat skrip — **membuktikan** semuanya dengan failure test, evidence, dan runbook.

1.3 Aturan main

Hal	Ketentuan
Peserta	**Individu.** Satu orang, satu repository, satu demo. Boleh berdiskusi dengan teman sekelas soal konsep dan cara debugging, tetapi kode, dokumen, dan angka di repository harus hasil kerjamu sendiri — dan kamu harus bisa menjelaskan setiap barisnya saat tanya-jawab.
Durasi	Dikerjakan **sendiri, bertahap sepanjang bootcamp**: tiap tahap terbuka setelah materi terkait diajarkan (lihat peta tahap). Tiga checkpoint pengumpulan + demo final di Grand Demo Day.
Stack	Stack *Your Big Data Lab* (compose yang sama; boleh ditambah service). Semua harus bisa dinyalakan dari nol di laptop lain dengan `docker compose up`.
Bahasa	Python 3.10+ dan SQL. Library bebas, tapi setiap dependency di-pin di `requirements.txt`.
Pengumpulan	Satu Git repository per peserta (GitHub). Dataset **tidak** di-commit (lihat §2). `.env` tidak di-commit.
Alat bantu AI	Boleh dipakai untuk belajar dan menulis kode, **wajib** dicatat di README bagian "Alat bantu" (apa yang dibantu, apa yang kamu ubah). Yang dinilai adalah pemahamanmu saat demo dan tanya-jawab, bukan siapa yang mengetik.

Hal	Ketentuan
Kejujuran data	Angka di README/dashboard harus berasal dari `outputs/` yang dihasilkan kode. Angka yang tidak bisa direproduksi = tidak ada.

Dokumen ini sengaja memberi **arah, bukan resep**. Tiap tahap punya target, kriteria lulus yang terukur, beberapa petunjuk, dan jebakan yang sering terjadi — tapi kodenya kamu yang tulis. Kalau ada yang ambigu, putuskan sendiri, tulis keputusanmu dan alasannya di README. Keputusan yang dijelaskan dengan baik dinilai lebih tinggi daripada keputusan "benar" yang tidak bisa kamu pertahankan.

2. Dataset

2.1 Mengunduh

Dataset: **Brazilian E-Commerce Public Dataset by Olist** di Kaggle (`olistbr/brazilian-ecommerce`), lisensi CC BY-NC-SA 4.0. Cara mengunduh:

1. Buat akun Kaggle, buka halaman dataset, klik **Download** (±45 MB zip), atau pakai CLI: `kaggle datasets download -d olistbr/brazilian-ecommerce`.
2. Ekstrak ke folder `data/raw/` di repository kamu (folder ini ada di `.gitignore`).
3. Jalankan `python scripts/check_dataset.py` dari starter: skrip ini memastikan sembilan file ada, nama kolomnya sesuai, dan jumlah barisnya sama dengan versi yang dipakai penilai. Kalau ada yang beda, berhenti dan tanyakan — jangan lanjut dengan data yang berbeda dari semua orang.

2.2 Sembilan file

File	Baris	Isi	Kunci yang kamu kira	Kenyataannya
`olist_orders_dataset.csv`	99.441	header order: status + 5 stempel waktu	`order_id`	benar
`olist_order_items_dataset.csv`	112.650	satu baris per **unit** barang	`order_id`	`(order_id, order_item_id)`
`olist_order_payments_dataset.csv`	103.886	pembayaran; bisa beberapa per order	`order_id`	`(order_id, payment_sequential)`
`olist_order_reviews_dataset.csv`	99.224	skor 1–5 + komentar (kebanyakan kosong)	`review_id`	**bukan** — cari tahu sendiri
`olist_customers_dataset.csv`	99.441	pelanggan per order	`customer_id`	ada dua id — mana yang "orang"?
`olist_sellers_dataset.csv`	3.095	seller + lokasi	`seller_id`	benar
`olist_products_dataset.csv`	32.951	produk + dimensi fisik	`product_id`	benar (perhatikan nama kolom)
`olist_geolocation_dataset.csv`	1.000.163	zip prefix → lat/lng	`zip_code_prefix`	**bukan** — satu prefix punya banyak baris
`product_category_name_translation.csv`	71	kategori Portugis → Inggris	`product_category_name`	benar (perhatikan byte pertama file)

Semua stempel waktu adalah waktu lokal Brasil tanpa zona (`YYYY-MM-DD HH:MM:SS`). Harga dalam BRL.

2.3 Yang perlu kamu tahu sebelum menyentuh apa pun

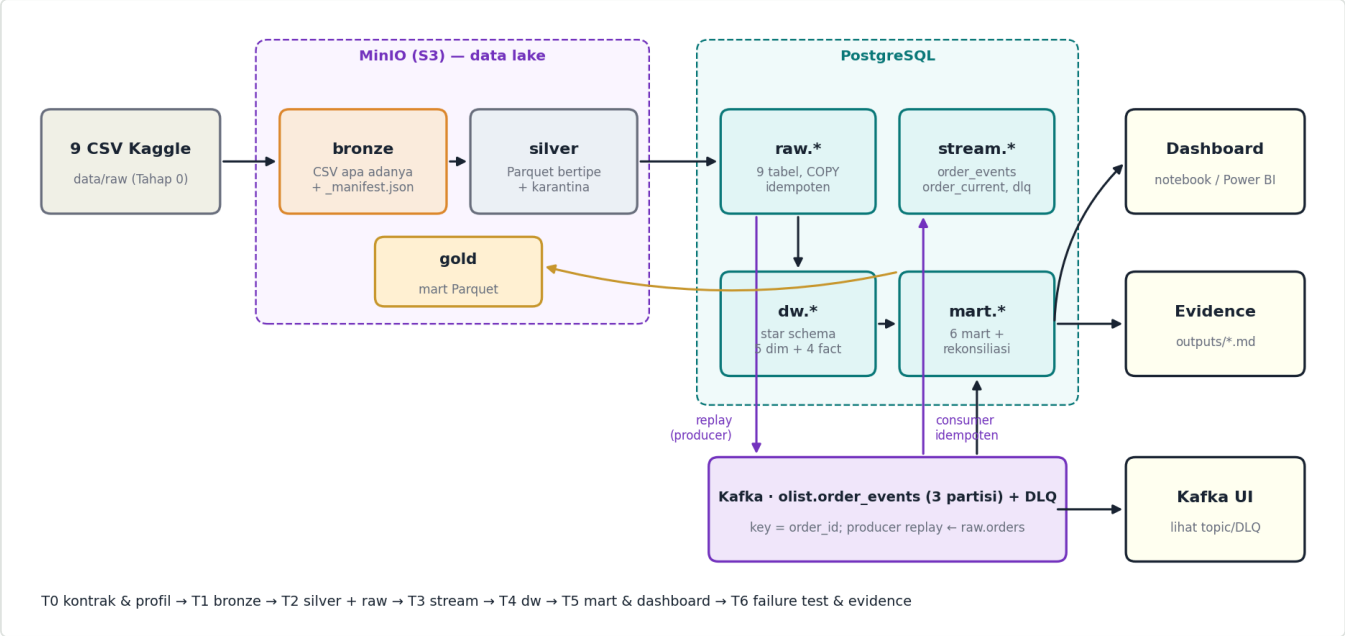
Dataset ini terkenal "bersih" — padahal tidak. Beberapa hal yang akan kamu temukan sendiri di Tahap 0 (kami sebutkan jenisnya, angkanya kamu yang cari):

- Dua jenis id pelanggan dengan makna berbeda. Salah pilih = jumlah pelanggan salah 3%.
- Kunci yang tidak unik di tabel yang "jelas-jelas" punya id.
- Kolom kode pos yang kehilangan angka nol di depan kalau dibaca sebagai angka — di **tiga** tabel.

- Koordinat yang jelas-jelas bukan di Brasil.
- Order yang tidak punya item, order yang tidak punya pembayaran, pembayaran bernilai nol.
- Status "delivered" tanpa tanggal terkirim; tanggal terkirim lebih awal dari tanggal diserahkan ke kurir.
- Total nilai item + ongkir **tidak sama** dengan total pembayaran — dan selisihnya bukan satu sebab.
- Bulan pertama dan bulan terakhir dataset hanya berisi segelintir order.
- Sebuah byte tak terlihat di awal salah satu file yang membuat join kategori diam-diam gagal.

Tidak ada satu pun dari ini yang boleh "dibersihkan" diam-diam. Setiap temuan harus punya **angka**, **keputusan**, dan **tempat keputusan itu diterapkan** di kode.

3. Arsitektur target



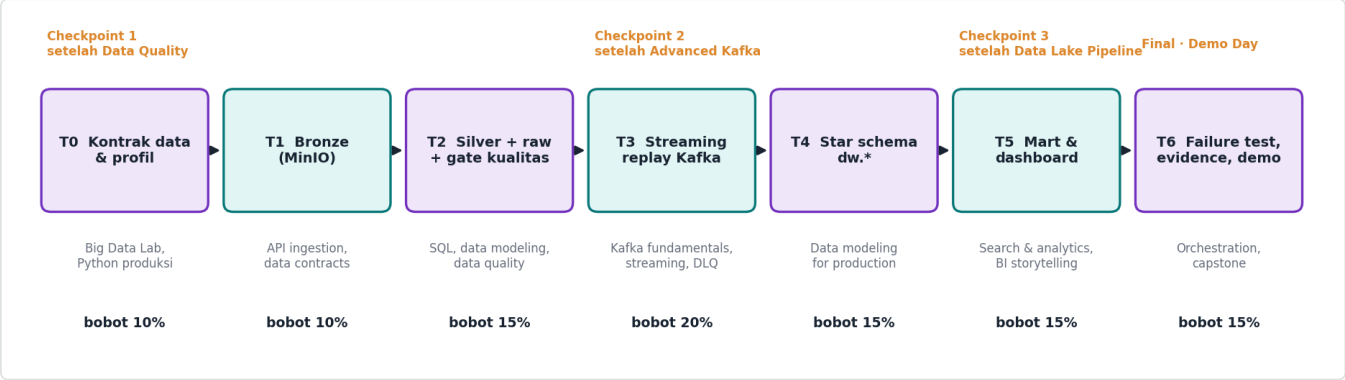
Arsitektur target: sumber → MinIO (bronze/silver/gold) → PostgreSQL (raw/stream/dw/mart) → dashboard; Kafka untuk replay event order

Lapisan yang harus ada (nama boleh beda, perannya tidak):

Lapisan	Tempat	Isi	Sifat
bronze	MinIO	file sumber apa adanya + manifest	append-only, bisa diaudit
silver	MinIO (Parquet)	data bertipe, sudah lewat gate kualitas, baris bermasalah dikarantina	bisa dibangun ulang dari bronze
raw	PostgreSQL	salinan silver untuk SQL	dimuat ulang idempoten
stream	PostgreSQL	hasil consumer Kafka: log event + status terkini per order	benar meski event dobel/terlambat
dw	PostgreSQL	star schema	grain jelas, PK/FK terbukti
mart (gold)	PostgreSQL + MinIO	tabel siap dashboard + rekonsiliasi	tiap angka bisa ditelusuri

Semua perintah dijalankan lewat **satu pintu**: `python pipeline.py <tahap>` (atau Makefile/justfile — terserah, yang penting satu). Penilai akan menjalankan perintah itu di laptop yang belum pernah melihat kodemu.

4. Peta tahap



Tujuh tahap, materi yang membukanya, bobot nilai, dan tiga checkpoint

Tahap	Nama	Dibuka setelah materi	Bobot	Dikumpulkan di
T0	Kontrak data & profil	Your Big Data Lab, Python for Production DE	10%	Checkpoint 1
T1	Bronze — ingest ke MinIO	Production-Grade API Ingestion, Multi-Source Ingestion & Data Contracts	10%	Checkpoint 1
T2	Silver + raw + gate kualitas	Advanced SQL, Data Modeling, Data Quality & Validation	15%	Checkpoint 1
T3	Streaming replay di Kafka	Kafka Fundamentals, Real Streaming Pipelines, Advanced Kafka	20%	Checkpoint 2
T4	Star schema (dw)	Data Modeling for Production (+ SQL)	15%	Checkpoint 3
T5	Mart & dashboard	Data Lake Architecture, Complete Lake Pipeline, Search & Analytics, BI	15%	Checkpoint 3
T6	Failure test, evidence, runbook, demo	Orchestration, Capstone Development	15%	Final
B1–B3	Bonus: Spark, Elasticsearch, Airflow	fase terkait	maks +15	Final

Kamu boleh mulai lebih awal dari jadwal pembukaan — semua tahap bisa dikerjakan dengan stack Pertemuan 9. Jadwal itu adalah batas **paling lambat** materi pendukungnya kamu dapatkan di kelas.

5. Tahap demi tahap

Format tiap tahap: **Target** (apa yang harus ada saat selesai) → **Kriteria lulus** (yang akan dicek penilai, satu per satu) → **Petunjuk** (arah, bukan resep) → **Jebakan** (kesalahan yang sudah pernah dibuat peserta sebelum kamu) → **Pertanyaan README** (yang harus kamu jawab tertulis).

T0 — Kontrak data & profil

Target. Kamu memahami dataset lebih baik daripada orang yang mengunggahnya ke Kaggle, dan pemahaman itu tertulis dalam `DATA_CONTRACT.md` yang menjadi acuan semua tahap berikutnya.

Keluaran wajib.

- `DATA_CONTRACT.md`: untuk tiap file — nama tabel yang kamu pakai, kolom + tipe target, kunci alami, jumlah baris yang diharapkan, dan **daftar bernomor temuan "kotor"** dengan angka dan keputusan penanganannya.
- `outputs/profile.md`: hasil profiling otomatis (null %, kunci unik atau tidak, rentang tanggal per kolom waktu, duplikat persis) — dihasilkan kode, bukan diketik.
- `python scripts/check_dataset.py` lulus.

Kriteria lulus.

- ☐ Sembilan tabel punya tipe target yang eksplisit (timestamp bukan teks, angka bukan teks, kode pos **teks**).
- ☐ Kunci alami tiap tabel disebut dan **dibuktikan** unik (atau dinyatakan tidak unik, dengan angkanya).
- ☐ Minimal **10 temuan kotor** dengan angka. Temuan tanpa angka tidak dihitung.
- ☐ Tiap temuan punya keputusan: karantina / biarkan & catat / tangani di dw / fallback — dan tahap mana yang menerapkannya.
- ☐ Profil dihasilkan ulang oleh satu perintah.

Petunjuk.

- Baca semua kolom sebagai teks dulu (`dtype=str`) saat profiling; konversi tipe adalah keputusan, bukan default library.
- Dua id pelanggan: hitung `nunique()` keduanya, lalu hitung berapa "orang" yang belanja lebih dari sekali. Angka itu yang akan menentukan grain `dim_customer` di T4.
- Untuk review: cek `review_id` unik atau tidak, lalu cek `order_id` unik atau tidak, lalu cek pasangannya.
- Untuk geolocation: hitung baris per zip prefix, duplikat persis, dan rentang lat/lng. Brasil kira-kira berada di lat -34…+5 dan lng -74…-35.
- Untuk file terjemahan: cetak `repr()` nama kolom pertama setelah dibaca dengan `encoding='utf-8'`. Lalu coba `utf-8-sig`.
- Hitung per order: Σ(price + freight) dari items vs Σ(payment_value) dari payments. Berapa order yang selisihnya > 0,01? > 1,00?
- Angka cek: orang unik **96.096**; order tanpa item **775**; order tanpa pembayaran **1**.

Jebakan.

- Membaca CSV dengan `pd.read_csv()` tanpa `dtype` lalu mengira kode pos "01003" memang "1003".
- Menghitung jumlah baris dengan `wc -l` — komentar review mengandung baris baru di dalam tanda kutip.
- Menulis "data sudah dicek, aman" tanpa satu pun angka.

Pertanyaan README. Dari semua temuan, mana yang kalau tidak ditangani akan membuat jawaban Q1–Q4 **salah secara diam-diam** (bukan error)? Sebutkan minimal tiga dan jelaskan mekanisme salahnya.

T1 — Bronze: ingest ke MinIO

Target. File sumber tersimpan di object storage **apa adanya**, dengan manifest yang membuktikan apa yang diunggah, kapan, dan isinya apa — dan perintah ingest aman dijalankan berulang.

Keluaran wajib.

- Bucket `bronze` berisi sembilan file di tata letak yang kamu tentukan (misalnya `olist/<tabel>/ingest_date=YYYY-MM-DD/<file>`).
- Satu manifest per tabel (JSON) berisi minimal: checksum isi (sha256/md5), ukuran byte, jumlah baris, nama sumber, waktu ingest, dan **riwayat versi**.
- `outputs/bronze_report.md`: tabel per file — status (`uploaded` / `unchanged` / `new_version`), baris, ukuran, waktu.

Kriteria lulus.

☐ Menjalankan ingest **dua kali** tanpa perubahan data → tidak ada upload ulang; laporan menyatakan `unchanged` untuk semua file.
☐ Mengubah satu byte di salah satu CSV lalu ingest → terdeteksi sebagai versi baru; versi lama **tetap ada** di bucket dan manifest.
☐ Manifest bisa dibaca tanpa kode kamu (JSON biasa), dan dari manifest saja seseorang tahu objek mana yang "terbaru".
☐ Ingest 9 file (±126 MB) selesai < 2 menit di laptop biasa.

Petunjuk.

- Hitung checksum **sebelum** memutuskan upload; bandingkan dengan versi terakhir di manifest. Itu seluruh rahasia idempotensinya.
- `boto3` bicara dengan MinIO lewat `endpoint_url`. Metadata objek (`Metadata={...}`) adalah tempat yang bagus untuk checksum, tapi manifest terpisah tetap perlu karena metadata tidak bisa di-*list* murah.
- Jangan baca 1 juta baris geolocation ke memori hanya untuk menghitung baris — hitung sambil streaming, atau pakai parser CSV yang memang paham tanda kutip.
- Bronze tidak mengubah apa pun: tidak rename kolom, tidak ganti encoding, tidak buang baris. Kalau tergoda, itu pekerjaan silver.

Jebakan.

- Menamai objek dengan tanggal hari ini saja → upload kedua di hari yang sama menimpa yang pertama.
- Menyimpan manifest di laptop (file lokal) alih-alih di bucket — laptop penilai (atau laptopmu yang lain) tidak punya.
- Menganggap "ukuran byte sama" = "isi sama".

Pertanyaan README. Kenapa bronze harus *append-only*? Beri satu skenario nyata di Olist di mana versi lama menyelamatkanmu.

T2 — Silver + raw + gate kualitas

Target. Data bertipe, divalidasi oleh aturan yang tertulis, baris bermasalah dikarantina (bukan dibuang), lalu dimuat ke PostgreSQL `raw.*` dengan cara yang aman diulang.

Keluaran wajib.

- Parquet silver per tabel di bucket `silver` (dan/atau cache lokal), dengan skema bertipe sesuai `DATA_CONTRACT.md`.
- **Gate kualitas**: minimal **10 aturan hard** (pelanggaran → baris dikarantina) dan **8 aturan soft** (dihitung, dilaporkan, baris tetap lewat). Tiap aturan punya nama, tabel, jenis, dan tindakan.
- Karantina tersimpan (Parquet/CSV) dengan kolom alasan.
- `outputs/dq_report.md`: angka tiap aturan, verdict per tabel (`PASS` / `PASS+KARANTINA` / `FAIL`).
- Schema `raw` di PostgreSQL berisi 9 tabel; `outputs/load_report.md` berisi jumlah baris dan durasi.

Kriteria lulus.

☐ Kolom waktu bertipe `timestamp`, uang `numeric`, kode pos `text` 5 digit di **semua** tabel yang punya kode pos.
☐ Jumlah baris `raw.*` = baris sumber - baris karantina, per tabel, dan angkanya ada di laporan.

☐ Ada **ambang** yang membedakan "kotoran biasa" (karantina lalu lanjut) dari "sumber rusak" (gate `FAIL`, tabel **tidak** dimuat). Ambangnya tertulis dan beralasan.
☐ Kolom hilang/berubah nama di CSV sumber → pipeline **berhenti sebelum** menyentuh Postgres, dengan pesan yang menyebut kolom mana.
☐ `load` dijalankan dua kali → jumlah baris sama (tidak dobel), dan kalau gagal di tengah, tabel tidak setengah terisi.
☐ Aturan kualitas punya unit test dengan data kecil buatan (tidak butuh Docker).

Petunjuk.

- Tipe target ditentukan dari kontrak, bukan ditebak per file: buat satu struktur data (dict/dataclass) yang dipakai profiling, silver, DDL, dan test. Kalau kontrak berubah, hanya satu tempat yang diubah.
- Aturan lintas tabel (orphan FK) butuh semua tabel di memori pada saat yang sama — rancang urutan bacanya.
- Postgres `COPY ... FROM STDIN` memuat 1 juta baris dalam hitungan detik; `INSERT` per baris butuh menit. `psycopg 3` punya `cursor.copy()`.
- `TRUNCATE` + `COPY` di dalam **satu transaksi** = idempoten dan atomik dengan dua kata kunci.
- Soft rule bukan hiasan: hasilnya adalah daftar keputusan untuk T4 (misalnya "547 order punya > 1 review → fact_review memilih yang mana?").

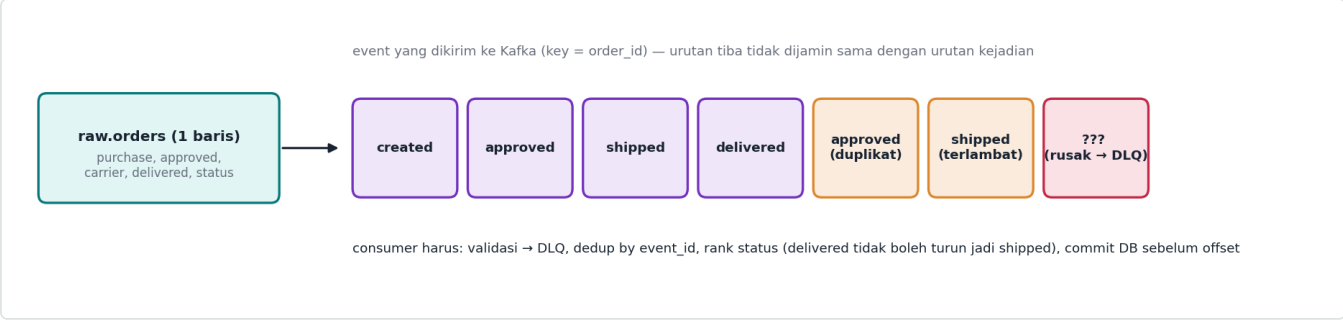
Jebakan.

- Membuang baris yang melanggar aturan tanpa menyimpannya: dua minggu kemudian ada yang bertanya "kenapa jumlah review kami turun?"
- Menjadikan "komentar review kosong" sebagai aturan not-null — 88% judul review memang kosong.
- Ambang karantina 0%: satu koordinat salah menghentikan seluruh pipeline. Ambang 50%: file yang separuhnya sampah lolos.
- Memuat `raw.*` dengan `INSERT` dari pandas (`to_sql`) tanpa indeks/`method` — 1 juta baris geolocation akan memakan 10+ menit.

Pertanyaan README. Angka ambang karantinamu berapa dan kenapa? Tunjukkan satu kasus yang lolos dan satu kasus yang gagal pada ambang itu.

T3 — Streaming replay di Kafka

Target. Satu *producer* memutar ulang order sebagai aliran event ke Kafka — **dengan gangguan yang disengaja** — dan satu *consumer* menyimpannya ke PostgreSQL sehingga status terkini tiap order **benar**, apa pun urutan dan berapa kali pun pesan tiba.



Satu order menjadi beberapa event; sebagian dikirim dobel, terlambat, atau rusak

Keluaran wajib.

- `producer`: membaca `raw.orders`, memecah tiap order menjadi event per perubahan status (dibuat, disetujui, diserahkan ke kurir, terkirim; plus status akhir seperti canceled/unavailable), mengirim ke topic dengan **key = order_id** dalam urutan waktu kejadian, dan **menyuntikkan gangguan terkontrol**: ±3% event dikirim dua kali, ±2% ditahan lalu dikirim terlambat, ±0,5% rusak (field hilang, format waktu salah, bukan JSON). Jumlah tiap gangguan dicatat ke file (ini "kunci jawaban" untuk consumer-mu sendiri).

- `consumer` dengan consumer group: validasi → pesan rusak ke **topic DLQ** (+ tabel) dengan alasan; event valid ke tabel log (`stream.order_events`, satu baris per event unik) dan tabel status terkini (`stream.order_current`, satu baris per order).
- Metrik per batch: jumlah pesan, baru, duplikat diabaikan, out-of-order, DLQ, dan **freshness** (`consumed_at − produced_at`, p50/p95).
- `outputs/stream_report.md`: membandingkan angka producer vs consumer, dan status terkini vs `raw.orders.order_status`.

Kriteria lulus.

- [ ] Event unik tersimpan = event unik dikirim (tidak ada yang hilang, tidak ada yang dobel).
- [ ] `stream.order_current.status` = `raw.orders.order_status` untuk **100%** order yang direplay.
- [ ] Duplikat diabaikan = duplikat dikirim; DLQ = pesan rusak dikirim. Angkanya cocok persis (atau selisihnya dijelaskan).
- [ ] Event yang datang terlambat **tidak menurunkan** status (delivered tidak kembali jadi shipped), tapi tetap tercatat di log.
- [ ] Consumer dimatikan paksa di tengah lalu dijalankan lagi → semua kriteria di atas tetap terpenuhi (tanpa reset).
- [ ] Replay 99.441 order selesai < 5 menit di laptop; freshness dilaporkan dengan angka.
- [ ] Consumer tidak pernah crash karena satu pesan jelek.

Petunjuk.

- Id event yang **deterministik** (misalnya gabungan `order_id` dan status) membuat "duplikat" bisa dikenali tanpa menyimpan apa pun selain event itu sendiri. Pikirkan apa yang terjadi kalau dua *replay* berbeda memakai id yang sama — itu fitur, bukan bug.
- Status punya urutan alami. Beri tiap status angka peringkat; status terkini hanya boleh naik. Pikirkan di mana `canceled` berada.
- Di PostgreSQL, `INSERT ... ON CONFLICT ... DO NOTHING` dan `ON CONFLICT ... DO UPDATE ... WHERE` adalah dua kalimat yang menyelesaikan sebagian besar masalah ini.
- Urutan commit menentukan segalanya: simpan ke database dulu, baru commit offset Kafka (`enable_auto_commit=False`). Lalu tanyakan: kalau proses mati **di antara** keduanya, apa yang terjadi saat restart? Jawabannya harus "tidak apa-apa".
- `send()` di kafka-python **tidak** mengirim; ia menaruh pesan di buffer dan mengembalikan *future*. Kalau broker mati cukup lama, future itu gagal diam-diam. Pikirkan siapa yang memeriksanya.
- Untuk melihat event beterbangan dan DLQ terisi: Kafka UI (`localhost:8080`). Untuk demo yang bisa diikuti mata, beri producer opsi kecepatan.
- Perhatikan: di data asli ada order yang tanggal terkirimnya lebih awal dari tanggal kurir. Replay yang jujur akan menghasilkan event "out-of-order" alami — consumer-mu harus tetap benar.

Jebakan.

- Dedup berdasarkan `order_id` saja → update status yang sah ikut dibuang.
- `auto_offset_reset='latest'` saat pertama kali jalan → consumer melewatkan semua yang sudah ada.
- Satu transaksi database per pesan → 400 ribu commit; replay 30 menit. Pikirkan batch.
- Menyamakan "late event" dengan "event dengan event_ts lama": yang penting adalah urutan **tiba** relatif terhadap status yang sudah tersimpan.
- Mengukur freshness dengan consumer yang baru dijalankan setelah producer selesai — angkanya mengukur berapa lama kamu menunggu, bukan pipeline.

Pertanyaan README. Jelaskan skenario crash consumer-mu langkah demi langkah: pesan ke-N sudah di Postgres, offset belum di-commit, proses mati. Apa yang terjadi saat restart, dan kode mana yang menjamin hasil akhirnya benar? Lalu: apa yang **tidak** idempoten di desainmu (petunjuk: DLQ) dan kenapa itu bisa diterima.

T4 — Star schema (dw)

Target. Model dimensional yang grain-nya tegas, kuncinya terbukti, dan keputusan penanganan "kotoran" dari T0/T2 benar-benar diterapkan di SQL.

Keluaran wajib.

- Schema `dw` berisi minimal: `dim_date`, `dim_customer`, `dim_product`, `dim_seller`, `dim_geo`, `fact_order` (header), `fact_order_item`, `fact_payment`, `fact_review`.
- SQL dalam file (bukan string di Python), dijalankan oleh satu perintah, idempoten (rebuild penuh boleh; jelaskan kenapa).
- PK dan FK **dipasang** (bukan hanya didokumentasikan).
- `outputs/dw_report.md`: jumlah baris per tabel + hasil pemeriksaan integritas (semua harus 0 baris bermasalah).
- Test terhadap database (pytest) untuk integritas dan grain.

Kriteria lulus.

☐ `dim_customer` satu baris per **orang** (angka cek: 96.096), bukan per order.

☐ `dim_geo` satu baris per zip prefix; koordinat di luar Brasil tidak ikut; pelanggan/seller yang kode posnya tidak ada di geolocation **tetap** punya FK yang valid (baris `unknown`).

☐ `dim_product` memakai nama kolom yang benar (typo sumber diperbaiki di sini) dan kategori Inggris dengan fallback untuk yang tidak punya terjemahan.

☐ `fact_order_item` tepat 112.650 baris; `fact_order` 99.441 baris termasuk order tanpa item dan tanpa pembayaran, dengan flag yang membedakannya.

☐ `fact_review` satu baris per order dengan **kebijakan pemilihan yang tertulis** untuk order yang punya > 1 review, dan kolom yang menyimpan berapa review yang sebenarnya ada.

☐ `is_late` bernilai **NULL** (bukan false) untuk order yang tidak/belum terkirim atau tanggal terkirimnya kosong; keterlambatan dihitung per tanggal.

☐ Semua FK terpenuhi 100% — dibuktikan oleh query yang mengembalikan 0, bukan oleh kalimat.

Petunjuk.

- `CREATE TABLE ... AS SELECT` lalu `ALTER TABLE ... ADD PRIMARY KEY` adalah test grain gratis: kalau grain salah, perintah kedua gagal.
- Atribut orang yang berubah antar order (kode pos, kota): putuskan ambil yang terbaru (SCD tipe 1) dan tulis itu. SCD tipe 2 tidak diwajibkan — tapi sebutkan apa yang hilang.
- Untuk `dim_geo`: agregasi 1 juta titik jadi ±19 ribu prefix. `avg()` untuk koordinat, `mode()` untuk kota/negara bagian.
- Bulan yang tidak lengkap: jangan dibuang; beri **flag** di `dim_date` agar mart dan dashboard bisa memilih.
- Denormalisasi secukupnya ke fact (status, tanggal, negara bagian) supaya mart tidak selalu join 4 tabel — tapi jangan sampai fact punya 60 kolom.

Jebakan.

- Join `customers` ke `orders` via `customer_id` (benar) tapi menghitung pelanggan dengan `customer_id` (salah 3.345 orang).
- `is_late = false` untuk order yang belum sampai → late rate terlihat lebih bagus dari kenyataan.
- Membandingkan `timestamp` terkirim dengan `date` estimasi tanpa menyamakan presisi — order yang sampai pukul 10 pagi di hari estimasi dianggap terlambat.
- Memasang FK ke `dim_geo` lalu menemukan 278 pelanggan tidak punya baris → "diperbaiki" dengan menghapus FK.

Pertanyaan README. Tulis kebijakan dedup `fact_review`-mu dan satu alternatif yang kamu tolak. Apa bedanya ke hasil Q2 (review vs keterlambatan)?

T5 — Mart & dashboard

Target. Empat pertanyaan manajemen terjawab dengan angka yang bisa ditelusuri, plus satu mart untuk feed status, plus rekonsiliasi yang jujur.

Keluaran wajib.

- Schema `mart` minimal: penjualan bulanan (bulan × kategori × negara bagian), performa pengiriman (bulan × negara bagian: terkirim, terlambat, late rate, rata-rata hari, skor review), scorecard seller, RFM pelanggan, freshness stream, dan **rekonsiliasi** (per order: nilai item vs dibayar, selisih, kelas penyebab).
- Mart diekspor ke bucket `gold` (Parquet) dan/atau CSV untuk dashboard.
- Dashboard: **Power BI** (`.pbix` tersambung ke Postgres) **atau** notebook di JupyterLab dengan grafik — minimal satu visual per pertanyaan + satu untuk freshness. Tiap visual punya satu kalimat *insight dengan angka*.
- `outputs/marts_report.md` : jawaban Q1–Q4 dalam tabel, dihasilkan oleh query yang **sama** dengan yang dipakai dashboard.

Kriteria lulus.

☐ Pendapatan memakai order dengan status yang kamu definisikan (tulis), dan dashboard bisa menyembunyikan bulan tidak lengkap.
☐ Late rate dihitung dari order yang `is_late` NOT NULL (penyebut jujur).
☐ RFM: skor F tidak boleh dihitung dengan kuintil mentah — pikirkan kenapa setelah melihat distribusi frekuensi.
☐ Rekonsiliasi: ≥ 98% order "cocok" dengan toleransi yang tertulis; sisanya diklasifikasikan (tanpa item, tanpa pembayaran, voucher, cicilan, tidak dijelaskan) — bukan disembunyikan.
☐ Setiap angka di dashboard bisa ditelusuri ke query → fact table (penilai akan memilih satu angka acak dan meminta kamu menelusurinya dalam 2 menit).
☐ Dashboard membaca dari Postgres/gold, **bukan** dari CSV mentah.

Petunjuk.

- Titik acuan "recency" bukan hari ini — datanya 2018.
- Review vs keterlambatan: bandingkan rata-rata skor untuk tepat waktu vs terlambat. Kalau selisihnya tidak mencolok, periksa `is_late` -mu.
- Seller scorecard: tetapkan jumlah order minimum sebelum membandingkan rasio; seller dengan 2 order dan 1 terlambat bukan "50% terlambat" yang berarti.
- Untuk Power BI: impor tabel `mart.*` lewat konektor PostgreSQL (`localhost:5432`); measure cukup `SUM` / `AVERAGE` karena agregasi berat sudah di SQL.
- Freshness: tabel metrik per batch dari T3 → agregasi per menit. Kalau lag tumbuh selama replay, itu temuan (consumer lebih lambat dari producer), tulis sebagai limitation + next experiment.

Jebakan.

- Menjumlahkan `payment_value` sebagai "pendapatan" — termasuk order yang dibatalkan dan order tanpa item.
- Mart yang hanya bisa dibangun kalau dashboard dibuka (logika di DAX/notebook, bukan di SQL yang bisa diuji).
- Grafik tanpa unit, tanpa periode, tanpa jumlah observasi.

Pertanyaan README. Dari rekonsiliasi: berapa total selisih uang yang tidak bisa kamu jelaskan, dan apa yang akan kamu tanyakan ke tim finance Olist?

T6 — Failure test, evidence, runbook, demo

Target. Kamu bisa menunjukkan — bukan mengklaim — bahwa platform ini menangkap kesalahan, pulih, dan bisa dijalankan orang lain.

Keluaran wajib.

- **Failure injection** otomatis, minimal **5 kasus**, masing-masing: apa yang dirusak, symptom yang terlihat (pesan/angka), verdict TERTANGKAP/LOLOS, mekanisme yang menangkap, cara pulih. Wajib ada: (a) kolom

hilang di sumber, (b) producer dijalankan dua kali, (c) consumer mati sebelum commit offset, (d) broker Kafka mati beberapa detik di tengah replay, (e) seluruh jalur batch dijalankan ulang tanpa perubahan. Hasil ke `outputs/failure_tests.md`.

- `EVALUATION_EVIDENCE.md`: durasi tiap tahap, jumlah baris tiap lapisan, hasil gate, hasil stream, integritas dw, jawaban Q1–Q4 ringkas, hasil failure test, **keputusan desain** (tabel: keputusan / alternatif yang ditolak / alasan), **limitation** (minimal 4), **next experiment** (minimal 3). Angka di dokumen ini **dibaca dari file hasil**, bukan diketik.
- `RUNBOOK.md`: cara menyalakan dari nol, urutan perintah, cara memeriksa tiap lapisan, 6+ symptom umum → pemeriksaan termurah → perbaikan, cara reset (dan apa yang hilang kalau reset).
- `README.md`: arsitektur, cara menjalankan dalam ≤ 10 perintah, jawaban semua "Pertanyaan README", bagian "Alat bantu".
- Demo 10 menit di Grand Demo Day (format di §8).

Kriteria lulus.

- ☐ Dari `docker compose down -v` sampai dashboard siap: ≤ 15 menit, ≤ 10 perintah, di laptop penilai.
- ☐ 5/5 kasus failure TERTANGKAP, dan stack pulih otomatis setelahnya (penilai akan menjalankan smoke/stream-report setelah inject).
- ☐ `pytest` hijau; ada test tanpa Docker (kontrak, aturan kualitas, logika event) dan test dengan Docker (integritas dw).
- ☐ Setiap angka di README cocok dengan `outputs/` (penilai mencocokkan tiga angka acak).
- ☐ Limitation jujur dan spesifik ("satu broker, tanpa replikasi" ya; "masih bisa ditingkatkan" tidak).

Petunjuk.

- Tulis failure injection sebagai fungsi yang **mengembalikan** verdict, bukan skrip yang mencetak. Yang mencetak adalah pelapornya.
- Untuk "consumer mati": jalankan consumer sebagai **proses terpisah** yang benar-benar dimatikan (exit code), bukan exception di proses yang sama — heartbeat Kafka dari proses yang masih hidup akan menahan partisi.
- Untuk "broker mati": `docker stop` + `docker start` dari kode, producer berjalan di thread lain. Ukur berapa pesan yang **gagal diam-diam** kalau future `send()` tidak diperiksa — lalu perbaiki.
- Evidence yang baik dibangun oleh satu perintah dari file JSON yang ditulis tiap tahap. Kalau kamu mengetik angka ke Markdown, dua minggu lagi angka itu bohong.

Jebakan.

- Failure test yang hanya membuktikan "ada exception" — yang dinilai adalah *tertangkap dengan pesan yang berguna* dan *pulih*.
- Runbook yang menyalin README. Runbook ditulis untuk orang yang sedang panik jam 2 pagi: symptom → periksa → perbaiki.
- Limitation yang sebenarnya adalah to-do list.

Pertanyaan README. Apa yang berubah dari desainmu kalau datanya 100× lebih besar dan ada SLA freshness 60 detik? Sebutkan tiga komponen yang pertama kali patah dan apa gantinya.

6. Bonus (maks +15, dinilai hanya jika T0–T6 lengkap)

Kode	Bonus	Yang harus ada	Nilai
B1	**PySpark** untuk silver → gold	Satu job Spark (batch) yang menghasilkan minimal dua mart yang sama dengan versi SQL; perbandingan durasi vs pandas/SQL pada tabel terbesar; tulis kapan Spark **tidak** layak di skala ini	+5
B2	**Elasticsearch** untuk pencarian produk & review	Index produk (kategori, dimensi) dan review (teks, skor); 3 query yang tidak enak di SQL (full-text, fuzzy, agregasi per kategori); 1 visual Kibana	+5
B3	**Airflow** DAG harian	DAG bronze → silver → load → model → marts dengan dependency, retry, dan satu **backfill** satu bulan; screenshot graph + log satu run gagal yang di-retry	+5

7. Deliverables & struktur repository

Struktur berikut adalah contoh, bukan keharusan — tapi penilai harus bisa menemukan semuanya tanpa bertanya.

```
olist-data-platform/
├── README.md                   # arsitektur, cara jalan, jawaban pertanyaan README, alat bantu
├── DATA_CONTRACT.md            # T0
├── RUNBOOK.md                  # T6
├── EVALUATION_EVIDENCE.md      # T6 (dibangkitkan)
├── docker-compose.yml  .env.example  requirements.txt
├── pipeline.py                 # satu pintu: init / up / check / profile / bronze / silver / load / stream /
model / marts / inject / report / down
├── <paket python>/             # satu modul per tahap
├── sql/                        # 00_schemas, dw/, mart/, checks/
├── tests/                      # tanpa Docker + dengan Docker
├── notebooks/                  # dashboard (kalau tidak pakai Power BI)
├── dashboard/                  # .pbix (kalau pakai Power BI)
├── scripts/check_dataset.py
├── data/raw/   (gitignore)     # CSV Kaggle
└── outputs/                    # profile, bronze_report, dq_report, load_report, stream_report, dw_report,
marts_report,
                                # failure_tests, dashboard/*.png, run_*.json
```

Pengumpulan tiap checkpoint = **tag Git** (`cp1`, `cp2`, `cp3`, `final`) + link repo di form kelas. Yang dinilai adalah isi tag itu.

8. Penilaian

8.1 Rubrik

Bobot mengikuti rubrik kurikulum; tiap aspek dinilai 0–4 lalu ditimbang.

Aspek	Bobot	4 — sangat baik	2 — cukup	0 — tidak ada
Functionality	25%	Semua tahap jalan dari nol dengan ≤ 10 perintah di laptop penilai; dashboard menjawab Q1–Q4; stream benar 100%	Sebagian tahap butuh intervensi manual; satu-dua kriteria lulus tidak terpenuhi	Tidak bisa dijalankan
Correctness	20%	Angka cek cocok; integritas 0; rekonsiliasi diklasifikasi; tidak ada "pembersihan diam-diam"	Ada angka yang meleset dan dijelaskan	Angka salah tanpa disadari
Engineering quality	20%	Satu pintu, kontrak terpusat, SQL di file, test tanpa & dengan Docker, idempoten terbukti	Kode jalan tapi duplikatif/manual; test minim	Skrip sekali pakai
Evaluation & reliability	20%	≥ 5 failure test TERTANGKAP + pulih; evidence dibangkitkan; limitation spesifik	Failure test ada tapi sebagian manual/tidak pulih	Tidak ada bukti
Documentation & explanation	15%	README/kontrak/runbook lengkap; semua pertanyaan README dijawab; keputusan bisa dipertahankan saat tanya-jawab	Dokumen ada tapi generik; sebagian pertanyaan tidak dijawab	Tidak ada

Nilai tahap (bobot di §4) dipakai untuk checkpoint; rubrik di atas dipakai untuk nilai akhir. Kedua-duanya memakai bukti yang sama: repository + demo.

8.2 Angka cek

Penilai memakai angka-angka ini (dan beberapa lagi yang tidak dibuka) untuk memeriksa pekerjaanmu dengan cepat. Kalau angkamu beda, bukan berarti salah — tapi harus bisa kamu jelaskan.

Apa	Angka
orang unik (`dim_customer`)	96.096
`fact_order_item`	112.650
`fact_order` (semua status)	99.441
order tanpa item / tanpa pembayaran	775 / 1
order yang punya ≥ 1 review	98.673
order delivered	96.478
late rate order delivered	≈ 6,5–7 % (tergantung definisi tanggal)
rata-rata skor review tepat waktu vs terlambat	≈ 4,3 vs ≈ 2,3
order dengan Σ item+ongkir ≠ Σ bayar (> 0,01)	ratusan, bukan ribuan
bulan "lengkap"	Jan-2017 … Agu-2018

8.3 Demo 10 menit (Grand Demo Day)

Menit	Isi
0–1	Masalah dan arsitektur dalam satu gambar
1–4	**Live**: `down -v` sudah dilakukan sebelum demo; jalankan pipeline sampai stream (boleh `--limit`), tunjukkan Kafka UI dan DLQ terisi
4–6	Dashboard: satu insight per pertanyaan, **dengan angka**; telusuri satu angka ke fact table

Menit	Isi
6–8	Satu failure injection live (pilih yang paling kamu banggakan)
8–10	Keputusan desain terpenting, limitation, next experiment
+ 5	Tanya-jawab: penilai memilih satu tahap secara acak dan satu potongan kode untuk kamu jelaskan baris per baris

9. Pertanyaan yang sering muncul

Boleh pakai library ingestion/orkestrasi siap pakai (dlt, Meltano, dbt)? Boleh untuk bonus. Untuk T0–T6 tulis sendiri — tujuannya kamu paham mekanismenya; nanti di pekerjaan kamu akan tahu kapan library itu membantu dan kapan menyembunyikan masalah.

Laptop saya 8 GB, stack berat. Pakai `docker-compose.lowmem.yml` dari lab (heap Kafka 384 MB) dan replay dengan `--limit`. Yang dinilai kebenaran, bukan kecepatan — tapi catat angkamu.

Dataset saya versinya beda (jumlah baris tidak cocok). Unduh ulang dari link resmi; kalau tetap beda, laporkan di grup kelas sebelum lanjut. Semua peserta harus memakai data yang sama supaya angka cek berlaku.

Apa yang dilakukan kalau stuck? Urutan termurah: baca error pertama (bukan terakhir) → `docker compose ps` → `docker compose logs --tail 50 <svc>` → periksa `.env` → baru tanya, dengan menyertakan ketiga hal itu. Pertanyaan "kok error ya?" tanpa symptom tidak akan dijawab.

Bolehkah menambah tahap atau mengubah urutan? Boleh, selama semua kriteria lulus tetap terpenuhi dan README menjelaskan kenapa.

Output bukan tujuan akhir. Kamu harus bisa menjelaskan asumsi, metric, trade-off, dan failure mode-nya. Keep building.